

הרצאה 9 – Node.js

בל הזכויות שמורות לקרן יעקב 2025

תוכן עניינים

1. [Node - Intro](#)
2. [Node - Modules](#)
3. [Node - Built-in Modules](#)
4. [Node - Npm Modules](#)
5. [Node - Custom Modules](#)



1. מבוא ל-Node.js

מבוא ל-Node.js

- Node.js היא סביבת צד שרת, סביבת הרצה של JS. יש לה סינטקס של JavaScript וכתובה ב- C++
- מבוסס על מנוע ה-JS של גוגל כרום גרסה 8 JavaScript Runtime V8 (נדבר בהמשך)
- Node.js מוסיף שכבה מלמעלה **המותאמת לריצה בצד שרת** ללא דפדפן ומאוד מהיר
- החל ב-2009 בפרוייקט קוד פתוח <https://github.com/nodejs/node>
- מבוסס מודולים הנכתבים על ידי הקהילה (NPM)
- גרסה נוכחית: 22.3.0, אבל גרסה 20.14.0 נתמכת לטווח ארוך (LTS)
- ב-Node.js מתווספות יכולות ופונקציונליות שאין ב-JS, למשל קוד לטיפול ב-streams.
- לא על כל שרת אחסון אפשר להריץ node, בקורס נשתמש ב-Render.

מבוא ל-Node.js - המשך

Branch: master ▾ node / src /

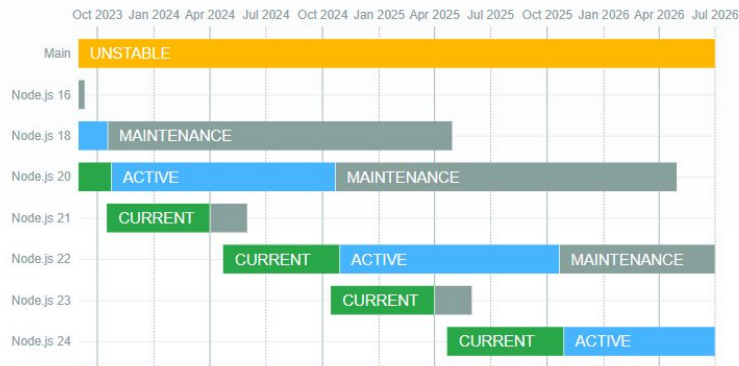
Create new file Upload files Find file History

devnexus and Trott src: fix asan build for gcc/clang ... Latest commit 4d2856e 27 days ago

..		
api	src: re-delete Atomics.wake	9 days ago
inspector	inspector: new API - Session.connectToMainThread	11 days ago
large_pages	src: add large page support for macOS	last month
res	build: update Windows icon to Feb 2016 rebrand	3 months ago
tracing	src: expose TraceEventHelper with NODE_EXTERN	2 months ago
js_native_api_v8.h	n-api: add APIs for per-instance state management	2 months ago
js_native_api_v8_internals.h	src: prefer v8::Global over node::Persistent	5 months ago
js_stream.cc	src: refactor uncaught exception handling	3 months ago
js_stream.h	src: remove unused method in js_stream.h	8 months ago
memory_tracker-inl.h	src: prefer v8::Global over node::Persistent	5 months ago
memory_tracker.h	src: prefer v8::Global over node::Persistent	5 months ago

Node versions

Release Schedule



Full details regarding Node.js release schedule are available on [GitHub](#).

Looking for latest release of a version branch?

Node.js Version	Codename	Release Date	npm	
v22.3.0	-	2024-06-11	v10.8.1	Releases Changelog Docs
v21.7.3	-	2024-04-10	v10.5.0	Releases Changelog Docs
v20.14.0	Iron	2024-05-28	v10.7.0	Releases Changelog Docs
v19.9.0	-	2023-04-10	v9.6.3	Releases Changelog Docs

<https://nodejs.org/en/about/previous-releases>

הגירסה המסומנת, בה נשתמש, היא
LTS = Long Term Support

V8 – JS Engine

המנוע V8 JavaScript engine נכתב בשפת C++ ותפקידו לקחת קוד של JavaScript ולהמיר אותו לקוד מכונה, וכך המחשב שלנו יוכל להריץ את אותו קוד.

המנוע מכיל תוכנה בשפת C++ לפי סטנדרט ECMAScript (גם Node כתוב ב- C++)

מהיר מאוד, עושה החלטות בזמן ריצה

V8 של גוגל הוא open source, פותח לצרכים של כרום אבל אפשר להשתמש בו גם עצמאית

הקוד ניתן להורדה: <https://chromium.googlesource.com/v8/v8/+master>

דוקומנטציה בגיטהאב: <https://v8.dev/docs>



JS בשפת שרת



● עם מה JS צריכה להתמודד כשפת שרת?

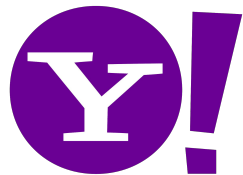
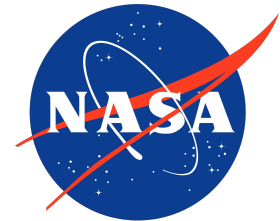
- יעילות ושימוש חוזר בקטעי קוד
- מהירות והתמודדות עם תהליכים שלוקחים זמן
- זמינות
- עומסים
- יכולת לתקשר על פני האינטרנט
- לקבל requests ולשלוח responses
- פונקציונליות - יכולת להתמודד עם קבצים
- פונקציונליות - יכולת להתמודד מול ה-DB



היכולות והשימושים של Node



Uber



PayPal



ebay[™]

- מהווה שרת ווב
- מהווה קוד בצד שרת
- עבודה עם דאטה חי, מהיר ולא רלציוני
- כל דבר שאפשר לכתוב ב-JavaScript (ולא תלוי בדפדפן)
- RESTful Services APIs
- משחקים מרובי משתתפים באמצעות sockets
- אפליקציות מבוססות ווב וענן
- בניית כלים וקוד ב-command line
- תקשורת ב-LIVE, chats

התכונות המרכזיות של Node



● א-סינכרונית

● Single Thread

● **Non-Blocking** - לא חוסם ריצה של תהליך אלא ממשיך לחכות לאירועים
מתאפשר בזכות thread pool, events queue ו-callbacks (לא נרחיב בקורס הזה)

● **Callback בשתי שורות:**

● פונקציה שאנחנו כותבים והיא תופעל רק כשהקוד יסיים את העבודה שלו / כשהתוצאה תהיה מוכנה.

● מאפשרת ל- Node להיות א-סינכרונית ו-single thread (גם Promises).

Node ממשיכה לקרוא קוד, וכשפעולה מבוצעת בהצלחה, היא נכנסת לפונקציה שהוגדרה ב-callback של הפעולה ומבצעת את הפקודות.

● דוגמה: טריגר לחיצה על כפתור ופונקציה אנונימית שעושה פעולה - callback.

REPL – הרצת קוד בטרמינל

- REPL = Read Eval Print Loop – קריאת פקודות, עיבודן והדפסתן
- ריצה ודיבאגינג במקביל
- כמו רוב השפות (Java, Python, PHP) – גם node מגיעה עם סביבה וירטואלית, a shell program, להרצת REPL,
- כך שאפשר לבדוק קוד תוך כדי הרצתו
- גם כרום מאפשר לעשות זאת ב- development tools שלו.

נפתח את הטרמינל ונרשום node כדי להיכנס לסביבה הוירטואלית של node

לאחר מכן נרשום את הפקודות ונראה את התוצאה:

```
Command Prompt - node

C:\Users\KerenY>node
Welcome to Node.js v20.14.0.
Type ".help" for more information.
> const x = 10, y = 20; x + y;
30
> _
```

Node basics & syntax



- מי שיודע JavaScript, יודע את הבסיס של NODE:
 - לא צריך להצהיר על סוגי המשתנים, נשתמש ב- `let`, `const`
 - מוסכמות: נקודה-פסיק בסוף ביטוי, `camelCase`, פסיקים להפרדת מפתח-ערך באובייקטים, הזחות להיררכיה, רווח לפני ואחרי סימנים כמו `=`
 - Node מבוסס אובייקטים ופונקציות
- תוספות ייחודיות ל- NODE:
 - **Buffer** - סוג משתנה חדש של Node, לאחסון מתקדם של חלקי streams או של קבצים
 - **מילים שמורות של השפה** (אובייקטים גלובליים): `require`, `process`, `global`, `exports`
 - **מבוסס מודולים**: גלובליים שמגיעים עם Node / שאנחנו יוצרים



Node.js -- ל-2 . מבוא Modules

Modules

- 
- מודול הוא קובץ או מספר קבצי JS שמאוגדים ומקושרים ביניהם בשביל שימוש חוזר – **reusable code**
 - אין השפעה ונגיעה של קוד ממודול אחד במודול אחר, עצמאיים.
 - המודול יכול להיות: (נרחיב בשקופיות הבאות)
 - מודול גלובלי שמגיע עם node
 - מודול מוכן שמורידים מ-npm
 - מודול אישי עצמאי custom made

Require

- require היא פונקציה (=אובייקט, הרי זה כמו JS) גלובלית (מזכירה את include ו-import) שמחזירה reference למודול
- require טוען את המודול הרצוי ובעזרתו אפשר לפנות לפונקציות שיוצאו ע"י exports מהמודול.
- ניתן לטעון json / package / module לתוך הקובץ הנוכחי בעזרתו, לוקאליים או מ-npm (נלמד בהמשך)
- דוגמה:

```
1 // Importing a local module
2 const myLocalModule = require('./path/myLocalModule');
3
4 // Importing a local json file
5 const jsonData = require('./path/filename.json');
6
7 // Importing a module from node_modules or Node.js built-in module
8 const crypto = require('crypto');
```

נתיבים של require



- הגדרה שגרתית של נתיב למודול:

1.	<code>require ('./my_md1_name');</code>	<code>// מצא את המודול בספרייה הנוכחית</code>
2.	<code>require ('../my_md1_name');</code>	<code>// מצא את המודול בספריית האב</code>
3.	<code>require ('./my_filename.json');</code>	<code>// מצא את קובץ הג'ייסון בתיקייה לוקאלית</code>
4.	<code>require ('express');</code>	<code>// node_modules מצא אותו בספריית</code>

- אם לא מציינים נתיב למודול (כמו בשורה הרביעית), Node מחפש את ספריית `node_modules` הקרובה ביותר
 - בהתחלה לוקאלית בספריית הפרוייקט
 - ואם לא – הוא עולה למעלה ומחפש בספריית היוזר של המחשב
 - ואם לא – הוא עולה למעלה ולמעלה
 - עד שהוא מגיע למודולים גלובליים הממוקמים בספריית `node_modules` ראשית במחשב.



3. מבוא ל-- Node.js Built-in modules

Built-in Node global modules **vs** built-in Node modules

- **Built-in Node global objects** – אובייקטים גלובליים שאפשר לגשת אליהם מכל מקום ולא צריך לייצא או

לטעון אותם, לא צריך להשתמש ב-require.

<https://nodejs.org/api/globals.html>

דוגמאות:

- Require
- Global
- Process
- Console
- Module

- **Built-in Node modules** – מודולים בסיסיים שמותקנים בספרייה העליונה של NODE על המחשב. כדי שתכנית תוכל להשתמש בהם נשתמש ב-require().

דוגמה Built-in Node global modules

```
Command Prompt - node
C:\Users\KerenY>node
Welcome to Node.js v20.14.0.
Type ".help" for more information.
> global
<ref *1> Object [global] {
  global: [Circular *1],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function: setImmediate] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function: setTimeout] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  queueMicrotask: [Function: queueMicrotask],
  structuredClone: [Function: structuredClone],
  atob: [Getter/Setter],
  btoa: [Getter/Setter],
  performance: [Getter/Setter],
  fetch: [Function: value],
  crypto: [Getter]
}
```

● נבנס לתהליך של Node ע"י כתיבת "node" בטרמינל
ונבקש את המודול המובנה global

Built-in Node modules (NOT global)



מודולים שמגיעים עם Node ומצריכים require

https://www.w3schools.com/nodejs/ref_modules.asp

דוגמאות:

- http ○
- url ○
- fs ○
- os ○

דוגמה לשימוש במודול os:

```
> os.version()
'Windows 10 Home'
> os.type()
'Windows_NT'
> _
```

נבנס לתהליך של Node ע"י כתיבת "node" בטרמינל
ונבקש את המודול המובנה os ונבחר אחת,
למשל os.type() שמחזירה את שם מערכת ההפעלה
או os.version() שמחזירה את גרסת מערכת ההפעלה



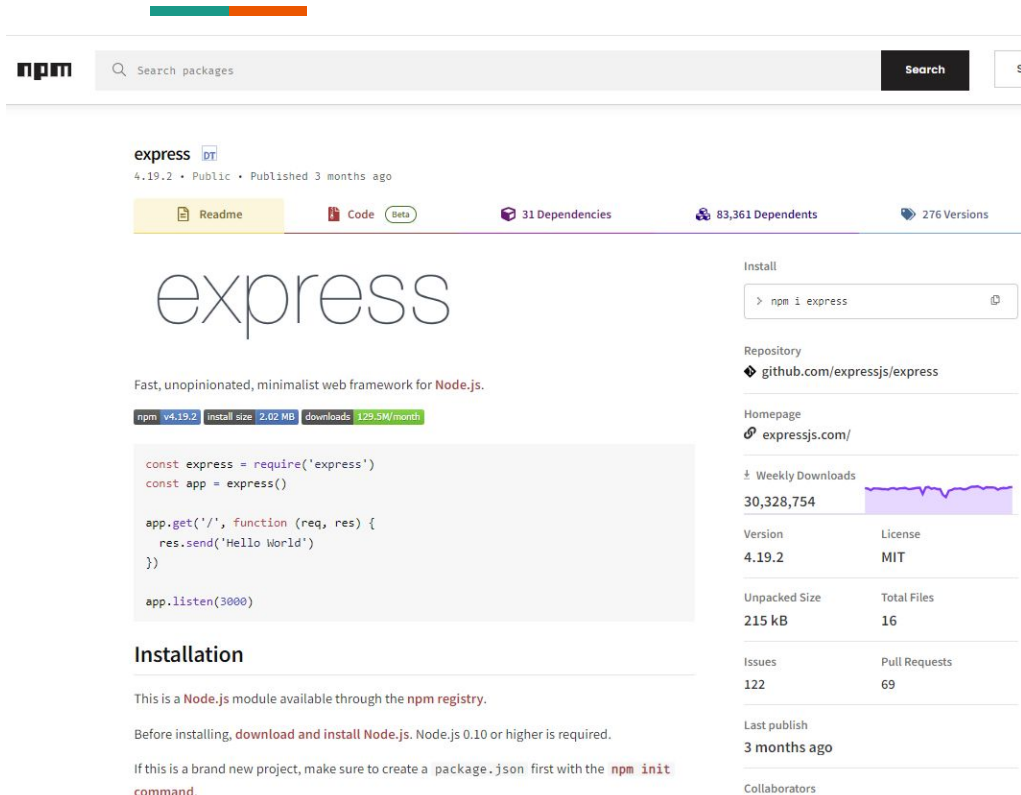
4. מבוא ל-- Node.js Npm modules



NPM

- חבילות מודולים: Node Package Modules – NPMs
ניתנים להתקנה בעזרת: **Node Package Manager – NPM**
- כדי לא לבלבל, לחבילות מודולים נקרא מודולים ולניהול התקנת מודולים נקרא **NPM**
- "חבילות מודולים" – ספריות קוד שניתן לשתף, להשתמש שוב ולהוסיף לפרויקטים שונים.
- NPM מותקן ביחד עם NODE
- אחד מהכוחות הגדולים של NODE הוא היכולת להתרחב, ניתן בקלות להוסיף חבילות מודולים
- דוגמאות לחבילות מודולים:
 - המודול Express מרחיב את יכולות העבודה על HTTP ב-NODE
 - המודול Nodemon מאפשר לנו לפתח ב-Node בלי לעצור ולהריץ את השרת בכל שינוי, הוא מזהה שינויים ומכיל אותם ברקע.

המשך - NPM



The screenshot shows the NPM website interface for the 'express' package. At the top, there's a search bar with 'Search packages' and a 'Search' button. Below the search bar, the package name 'express' is displayed with a version '4.19.2', 'Public' status, and 'Published 3 months ago'. It also shows '31 Dependencies', '83,361 Dependents', and '276 Versions'. The main section features the 'express' logo and the tagline 'Fast, unopinionated, minimalist web framework for Node.js.' Below this, there's a code block showing a simple Express application setup. The 'Installation' section states that this is a Node.js module available through the npm registry and provides instructions on how to install it. On the right side, there's a sidebar with an 'Install' section showing the command 'npm i express', a 'Repository' section linking to 'github.com/expressjs/express', a 'Homepage' section linking to 'expressjs.com/', a 'Weekly Downloads' section showing '30,328,754' downloads, a 'Version' section showing '4.19.2' and 'License' 'MIT', a table with 'Unpacked Size' (215 kB) and 'Total Files' (16), an 'Issues' section showing '122' and 'Pull Requests' '69', a 'Last publish' section showing '3 months ago', and a 'Collaborators' section.

express DT

4.19.2 • Public • Published 3 months ago

Readme Code Beta 31 Dependencies 83,361 Dependents 276 Versions

express

Fast, unopinionated, minimalist web framework for Node.js.

npm v4.19.2 install size 2.02 MB downloads 129.5M/month

```
const express = require('express')
const app = express()

app.get('/', function (req, res) {
  res.send('Hello World')
})

app.listen(3000)
```

Installation

This is a **Node.js** module available through the **npm registry**.

Before installing, **download and install Node.js**. Node.js 0.10 or higher is required.

If this is a brand new project, make sure to create a `package.json` first with the **npm init** command.

Install

```
> npm i express
```

Repository

github.com/expressjs/express

Homepage

expressjs.com/

Weekly Downloads

30,328,754

Version	License
4.19.2	MIT

Unpacked Size	Total Files
215 kB	16

Issues	Pull Requests
122	69

Last publish

3 months ago

Collaborators

[/https://www.npmjs.com](https://www.npmjs.com)

דוגמה מתוך npm לספריה express מצד שמאל.

אפשר לראות דוגמה לשימוש, ומצד ימין פרטים על הספריה הזו, כמו מספר הורדות שבועי, גרסה, מתי עודכנה בפעם האחרונה... אילו פרטים שנסתכל עליהם כשנרצה להשתמש בספריה כלשהי.

The global scope

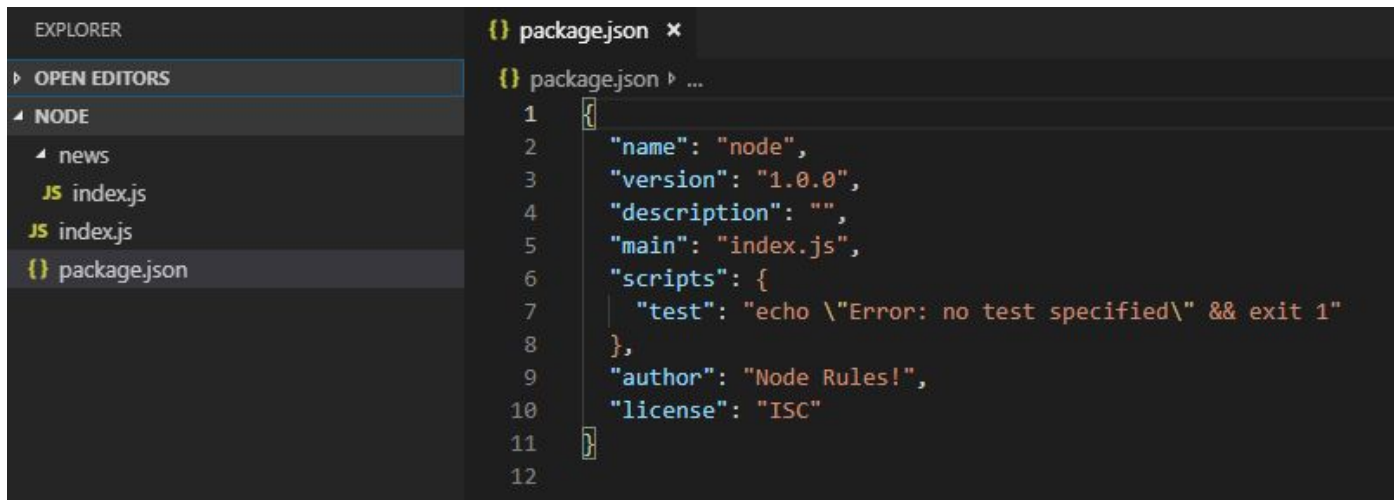
- 
- בדפדפן, JS באופן אוטומטי מגדיר הכל ב- global scope, כלומר אם לא נגדיר משתנה הוא יהיה גלובלי
NODE תוכננה להתנהג הפוך: לוקאלי כדיפולט
 - לכן אם רוצים להתקין מודול כגלובלי צריך לציין זאת מפורשות באמצעות g- בזמן ההתקנה,
לא מומלץ כשלא חייבים.
 - כדיפולט, מודולים מותקנים לוקאלית בספריה שבה מריצים את פקודת התקנת המודול
 - אם פרויקט אחד עובד עם מודול X גרסה 1, ופרויקט אחר עובד גם עם מודול X אבל גרסה 2, יהיה העתק של כל
גרסה בכל פרויקט (מניעת התנגשויות), כך NODE מתוכנן לעבוד.

package.json

- בכל פרויקט שבו מתקינים מודולים מ-NPM, יש להגדיר אותם ב- "הוראות ההתקנה של הפרויקט", הן מפורטות בקובץ package.json
- כל פרויקט חדש צריך להיות מאותחל על ידי NPM, ע"י הפקודה `npm init` והיא זו היוצרת את הקובץ package.json
- בקובץ מפורטים בו פרמטרים שרלוונטיים להתקנת הפרויקט, כולל מודולים, תתי-מודולים
- נעזר בו בהתקנת פרוייקטים חדשים - על סמך ההוראות שבקובץ, הוא יתקין לנו את כל המודולים בפקודת "התקן". זו הסיבה שאף פעם לא נעלה את תיקיית **node_modules לשרת !!!**
- בנוסף, כל חבילת מודול מגיעה עם package.json שמתאר את החבילה:
 - שם המודול
 - גרסה
 - שם המתכנת
 - Dependencies
 - ועוד

Npm init & package.json

- נרשום ב-cmd את הפקודה npm init כשנרצה ליצור קובץ package.json בתחילת הפרוייקט
- תוצג רשימת שאלות, נקבל הכל, רק נגדיר את שם ה-author, השאר לא קריטי
- y-npm init יחזיר yes להכל
- קובץ package.json ייוצר בתיקייה עם ההגדרות שהכנסנו
- אם יש כבר קובץ js בתיקייה, הוא ירשם בתור הקובץ המפעיל main:index.js



```
{} package.json x
{} package.json ▶ ...
1  {
2    "name": "node",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "test": "echo \"Error: no test specified\" && exit 1"
8    },
9    "author": "Node Rules!",
10   "license": "ISC"
11 }
12
```

Npm install



נרצה להוסיף מודול חדש לפרוייקט, לדוגמה express שנביר בהמשך

<https://www.npmjs.com/package/express>

מתקינים את המודול ומוסיפים הוראת שמירה

בך התלות בשימוש במודול זה נשמרת בקובץ ה- package.json:

```
$ npm install express
```

נוצרה ספריה (אם לא היתה קיימת קודם): node_modules ליד package.json.

ונוצר קובץ חדש package-lock.json

הקבצים והתיקיות נוצרו בפרוייקט שלנו- כל ה dependencies ש-express צריכה

package-lock.json

```
1 package-lock.json x
2 {} package-lock.json > ...
3 1
4 "name": "node",
5 "version": "1.0.0",
6 "lockfileVersion": 1,
7 "requires": true,
8 "dependencies": {
9   "abbrev": {
10     "version": "1.1.1",
11     "resolved": "https://registry.npmjs.org/abbrev/-/abbrev-1.1.1.tgz",
12     "integrity": "sha512-nne9/IQ/hz1hY6pdDnbBtz7DjPTKrY00P/zvPSm5oFk16xvGrGnXn/VeTNNfntAf29/1RtehszU99qTii0A==",
13     "dev": true
14   },
15   "accepts": {
16     "version": "1.3.7",
17     "resolved": "https://registry.npmjs.org/accepts/-/accepts-1.3.7.tgz",
18     "integrity": "sha512-0e966a5E956P4QZC954y9qAbglbRhl6U4vKn5AgpB96bdS4f21nuJ6yZCzWqTj5xRUJfBacVZQJ8xV5Q==",
19     "requires": {
20       "mime-types": "~2.1.24",
21       "negotiator": "0.6.2"
22     }
23   },
24   "ansi-align": {
25     "version": "2.0.0",
26     "resolved": "https://registry.npmjs.org/ansi-align/-/ansi-align-2.0.0.tgz",
27     "integrity": "sha1-w2rsy6VjuJzrVw82kPCx2eNUf38=",
28     "dev": true,
29     "requires": {
30       "string-width": "^2.0.0"
31     }
32   },
33   "ansi-regex": {
34     "version": "3.0.0",
35     "resolved": "https://registry.npmjs.org/ansi-regex/-/ansi-regex-3.0.0.tgz",
36     "integrity": "sha1-7QMXwyIGT3lGAKWa922Bas32Zg=",
37     "dev": true
38   },
39   "ansi-styles": {
40     "version": "3.2.1",
41     "resolved": "https://registry.npmjs.org/ansi-styles/-/ansi-styles-3.2.1.tgz",
42     "integrity": "sha512-069231538693848309399076907810933262572435f5f515266570212f17d0e",
43     "dev": true,
44     "requires": {
45       "color-convert": "^1.9.0"
46     }
47   }
48 }
49 }
```

<https://docs.npmjs.com/files/package-lock.json>

מכיל את כל רשימת ה- dependencies

הקובץ מתווסף מאז גרסה 5 של npm

בגלל ש-Node תלויה כל כך במודולים, המודולים ברובם לוקליים בספריית הפרויקט, ובגלל שיש המון מודולים, כדי למנוע התקנות של מודולים ישנים או קונפליקטים

לא ניגע בקובץ זה!

NPM Packages – dependencies **vs** dev-dependencies

- 
- ה - package.json מחזיק את המודולים שהפרויקט שלנו משתמש בהם - dependencies
הוא גם מחזיק את הכלים (המודולים) הנדרשים על מנת לפתח את הפרויקט שלנו והם נמצאים ב -dev-dependencies
 - מה ההבדל בין dev-dependencies ל- dependencies ?
 - בשנרצה להוסיף מודול לפרויקט נשתמש בflag המתאים:
 - -D / --save-dev,
 - -g / --global
 - בשנקבל פרויקט ונרצה להתקין את כל ה-dependencies שלו נריץ -
 - npm install

NPM Packages – global **vs** local



- מתי נשמור גלובלי ומתי לוקאלי ?
 - לוקאלי – כשנרצה להשתמש במודול בפרוייקט שלנו ע"י `require()`
 - גלובלי – כשנרצה להשתמש במודול דרך ה-`cmd`, כך שהקבצים יהיו במחשב

● מתי נשמור לוקאלית ב- `dev-dependencies` ? כשנצטרך את המודולים רק בזמן הפיתוח

● מתי נשמור לוקאלית ב- `dependencies` ? כשנצטרך את המודולים בזמן הריצה



סיכום – Installing Npm modules



- בשמתקינים מודול מ- NPM:
- \$ npm install some_md1
- נוצרת ספריית node_modules ובתוכה המודול שירד מהרשת:
- Users/me/my_app/**node_modules**/some_md1/
- ובקובץ המפעיל נרשום את הקוד שקורא למודול:
- require ('some_md1');



Node.js --ל מבוא .5

Custom modules

הבנת מודולים

```
var func1 = function() {...};  
var func2 = function() {...};  
var func3 = function() {...};
```

```
exports.func1 = func1;  
exports.func2 = func2;
```



```
Public:  
func1, func2
```

```
Private:  
func3
```



```
var mdl1 = require('./mdl1');  
mdl1.func1();  
mdl1.func2();
```

דוגמה – Custom module



- בבניית מודול משלנו, יהיו לנו 2 קבצים:

```
JS index.js x ... JS mdl.js x
JS index.js JS mdl.js
1 require("./mdl.js"); 1 console.log("Hello World");
```

קובץ מפעיל

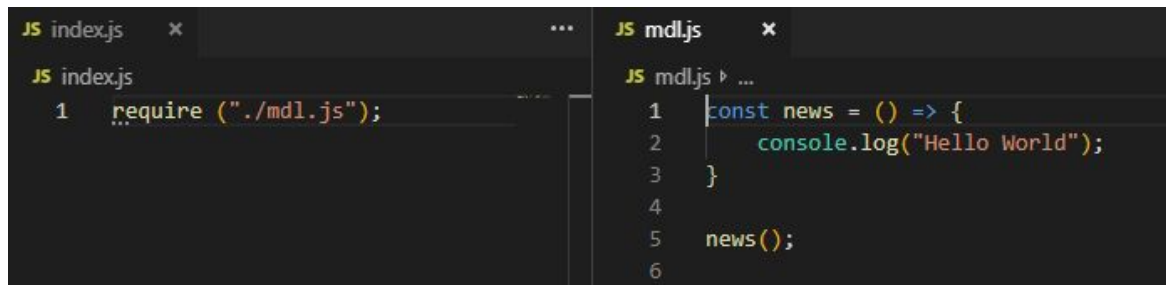
קובץ מודול – מופעל

- require מקבל פרמטר אחד: נתיב
- require(./) – הנתיב הוא הספרייה הנוכחית
- require מחזיר reference למודול שעשו לו export בקובץ המופעל
- בדוגמה הזו אנו לא צריכים reference אלא לטעון את המודול
- בעמודים הבאים נבין מה זה אומר שאין השפעה ונגיעה של קוד ממודול אחד במודול אחר, המודולים עצמאיים

דוגמה המשך – Custom module

נוסיף במודול פונקציה אנונימית וקריאה לפונקציה מהמודול

נריץ בטרמינל



```
JS index.js x ... JS mdl.js x
JS index.js
1 require("./mdl.js");

JS mdl.js > ...
1 const news = () => {
2   console.log("Hello World");
3 }
4
5 news();
6
```

קובץ מפעיל

קובץ מודול – מופעל

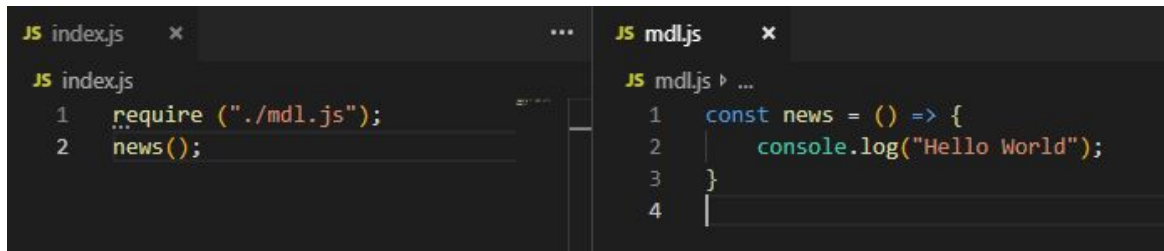


```
Administrator: Windows PowerShell
PS C:\Users\OWNER\Desktop\Node> node index.js
Hello World
PS C:\Users\OWNER\Desktop\Node>
```

דוגמה המשך - Custom module

- נוסף בקובץ המפעיל קריאה לפונקציה שבמודול

- הרצה בטרמינל - תעבוד ?



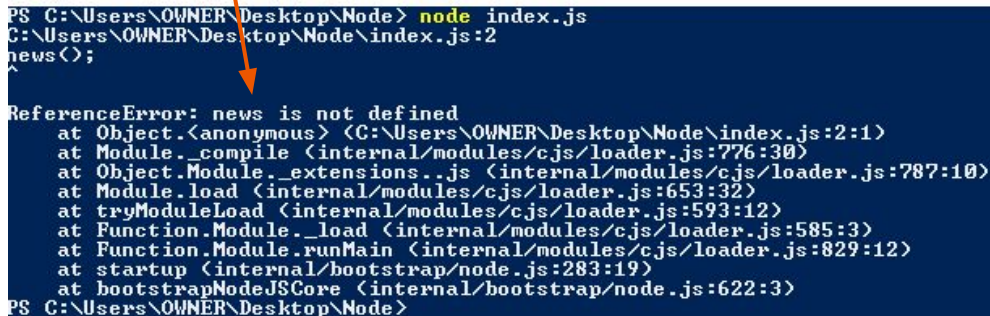
```
JS index.js x
JS index.js
1 require("./mdl.js");
2 news();

JS mdl.js x
JS mdl.js ▸ ...
1 const news = () => {
2   console.log("Hello World");
3 }
4
```

קובץ מפעיל

קובץ מודול - מופעל

Administrator: Windows PowerShell



```
PS C:\Users\OWNER\Desktop\Node> node index.js
C:\Users\OWNER\Desktop\Node\index.js:2
news();
^
ReferenceError: news is not defined
    at Object.<anonymous> [C:\Users\OWNER\Desktop\Node\index.js:2:1]
    at Module._compile (internal/modules/cjs/loader.js:776:30)
    at Object.Module._extensions..js (internal/modules/cjs/loader.js:787:10)
    at Module.load (internal/modules/cjs/loader.js:653:32)
    at tryModuleLoad (internal/modules/cjs/loader.js:593:12)
    at Function.Module._load (internal/modules/cjs/loader.js:585:3)
    at Function.Module.runMain (internal/modules/cjs/loader.js:829:12)
    at startup (internal/bootstrap/node.js:283:19)
    at bootstrapNodeJSCore (internal/bootstrap/node.js:622:3)
PS C:\Users\OWNER\Desktop\Node>
```

- קיבלנו שגיאה, למה ?